Copyright 2026 David Gerald Morey - Protovelop

List of Requirements:

VS Code
VS Code Pic64GX SDK extension
VS Code Serial Monitor extension (not required but highly recommended)
Python v 3.8 - 3.12 (3.12 recommended, DO NOT USE HIGHER VERSION), recommend to use a ven
v in the project directory for this version
CMake
WSL 2

PIC64GX BUILD HOW TO

Note: Pic64gx requires python 3.12 - recommended to setup a venv for this python version
will also need vscode and downloaded pic64gx sdk

UNDERSTANDING THE ARCHITECTURE:

PIC64GX is a 64 MPU with 5 different hart cores, one of which can run Linux - named hart
0 which is an e51 type of core - and 4 which can run c applications bare metal, and which
 are cores of type u54. The PIC can be run in the default Symmetric Multiprocessing (SMP)
 mode which uses identical processors for all tasks, which can be inefficient due to exce
ssive resource allocation for certain workloads, or it can be configured for Asymmetric M
ulti Processing (AMP) operation, where various processing contexts are separated and opti
mized for specific tasks. SMP utilises a shared memory structure whereas AMP has memory p
artitions specifically for each core. In addition to this AMP can be configured to run a
Linux operating system on one core and either a bare metal or RTOS application on the oth
er cores. Bare metal configurations operate without an operating system directly interact
ing with hardware through memory mapped registers, and is usually suitable for low cost/
low complexity applications where real time deadlines are not critical. Real Time Operati
ng Systems (RTOS) on the other hand, can be configured for deterministic operation, where
 a guaranteed time frame for code execution and response to certain events is needed. RTO
S systems are much complicated and introduce high memory usage as well as debugging chall
enges if not well understood.
The first thing to decide is which of the 3 operational configurations are needed, as thi
s will determine how the SDK is used.

For the Hart 0 e51 core, a Linux image is needed. Canononical have a prebuilt Linux image
 for the PIC64GX, but it is a full scale headless operating system, meaning it has a lot
of general purpose drivers and setups that that make booting up take quite some time. To
build a custom Linux OS with only the needed drivers and packages that boots much quicker
, either buildroot or Yocto need to be used. Buildroot is a lot easier but Yocto provides
 much more customization at a cost of complexity.
Then comes the problem of taking the c source code placed in the hart core folders of the
 project and actually burning into the PIC64GX's on board memory so it runs independently
 of a debugging environment connected to the host computer. The method by which this is d
one is to use Hart Software Services (HSS) and the xpack-riscv-none-elf-gcc-13.2.0-2-win3
2-x64 toolchain to boot payloads containing the hart configurations: Yocto/buildroot is u
sed to create a uboot.bin file whilst the RISCV toolchain is what is used to create a pay
load yaml using both the {project_name}.elf and {project_name}.bin files created after bu
ilding the project (look in the project's "build" folder). An ELF file is itself a machin
e binary file that contains your own compiled memory locations and code.
It requires both a .c source file and an .ld linker file to create - the c soure file sho
uld be what ever is in the {project_name} hart folders, whilst the linker should be found
 in the {project_name} build folders after building.
A linker (.ld) file tells the compiler where everything lives in memory; it tells the lin
ker to "put this file at this memory address" look at examples to get an idea of how the

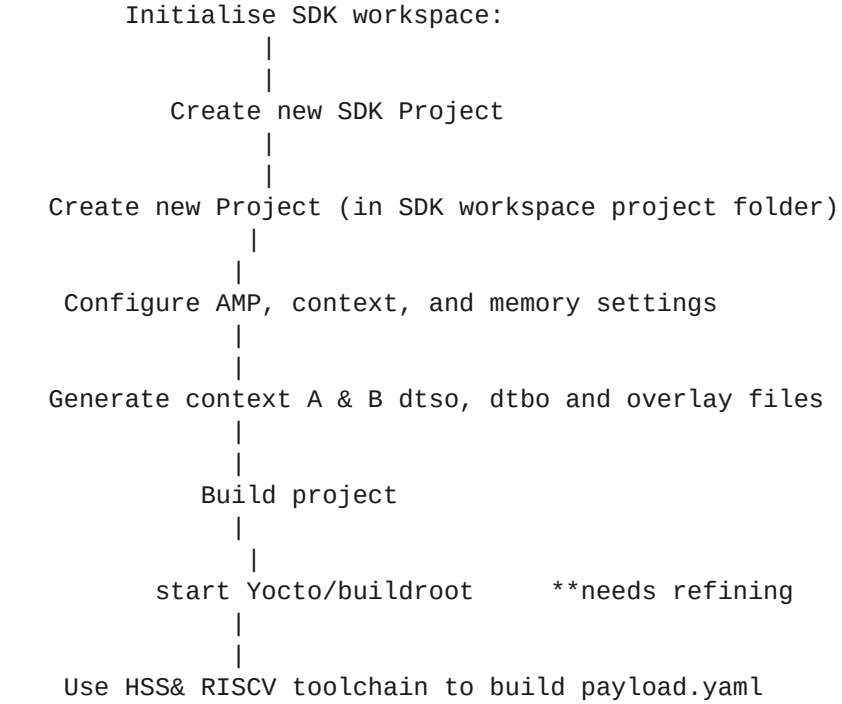
linker file should be written.

typical linker file:

```
        MEMORY
    {
      RAM (rwx) : ORIGIN = 0x08000000, LENGTH = 64K
    }

    SECTIONS
    {
      . = 0x08000000;

      .text : {
        *(.text*)
      }

      .rodata : {
        *(.rodata*)
      }

      .data : {
        *(.data*)
      }

      .bss : {
        *(.bss*)
      }

      . = ALIGN(8);
      _stack_top = . + 0x1000;
    }
```

Buildroot/ Yocto are supposed to be built on a Linux host computer running ubuntu due to the file system commands; it uses case sensitive commands unlike Windows, which will cras h the system if trying to build natively on Windows. It is for this reason that if trying to cross compile on a Windows environment, builroot/Yocto must be used in a virtual Linu x environment like WSL2 (recommended) or Virtual Box. NOTE that The HSS RISCV toolchain compilation is done as a secondary step of the Yocto/buildroot process.

This give a SDK new project workflow for bare metal and AMP operation along the lines of:



|
|

BUILDROOT:
Make sure buildroot and busy box have been cloned into the workspace directory:

```
git clone https://github.com/linux4microchip/buildroot-external-microchip.git
git clone https://git.busybox.net/buildroot -b 2025.02
```

Change to the buildroot directory:

```
cd buildroot
```

Use the appropriate defconfigs (check that these are the correct ones):

```
BR2_EXTERNAL=../buildroot-external-microchip/ make icicle_defconfig
```

Make the buildroot:

```
make
```

The resulting bootloader, kernel, and root filesystem will be put in the 'output/images' directory. There is also a complete sdcard.img.

To configure userspace packages and build:

```
make menuconfig
make
```

To configure the kernel and build:

```
make linux-menuconfig
make
```

Create a list of software licenses used:

```
make legal-info
```

There are several configurations available for PIC64GX Curiosity Kit. To generate an imag e, choose any of the PIC64GX Curiosity Kit defconfigs provided in the configs directory a nd follow the corresponding instructions.

For example, to build an image suitable for programming to the SD card for the PIC64GX Cu riosity Kit:

```
BR2_EXTERNAL=../buildroot-external-microchip/ make pic64gx_curiosity_kit_defconfig
make
```

Create an Image for SD Card:
An image is generated in the file sdcard.img in the output/images directory. The first pa rtition of this image contains a U-Boot binary, embedded in a Hart Software Services (HSS) payload. The second partition contains a FAT filesystem with a U-Boot env and an ITB fi le containing the kernel and the device tree. The third partition contains the file syste m. This image can be written directly to the SD card.

The pic64gx_curiosity_kit_defconfig generates an image containing a root filesystem, wher eas the pic64gx_curiosity_kit_initramfs_defconfig generates an image with a RAM-based fil esystem.

There are several ways to copy the image to an SD card:

a) Copy the image to the SD card using the standard Unix dd tool:

Find the device node name for your card and then copy the image as shown below:

```
cd output/images
sudo dd if=sdcard.img of=/dev/sdX bs=1M
```

b) Copy the image to the SD card using bmaptool (recommended)

This is a generic tool for creating a block map (bmap) for a file and copying files using this block map. Raw system image files can be flashed a lot faster with bmaptool than with traditional tools, like "dd".

```
cd output/images
sudo bmaptool copy sdcard.img /dev/sdX
```

For your SD card, you need at least 8GB. All the data on the SD card will be lost.

IMPORTANT: MAKE SURE PIC64GX SDK AND HART SOFTWARE SERVICES ARE INSTALLED AND ADDED TO PATH. Current installation is in c:/users/fender{subfolder}

1. VS Code should be installed along with the PIC64GX SDK and serial monitor extensions. Whilst not strictly needed, as alternatives like Putty can be used instead, the serial monitor extension is recommended as it gives the opportunity to monitor three different UART com ports at once, so it can be set up to debug the com ports used by the PIC64GX. Putty is also recommended as it makes editing Linux's nano txt editor easier with copy and pasted commands from a windows host pc, though is only useful after SSH has been set up. The SDK on the other hand is non negotiable as it contains the drivers needed for these com PIC64GX com ports to become operational.
IMPORTANT: Ensure a decent high current USB type C cable is connected to the PIC64GX curiosity kit, and that the Type a side connects to a USB 3 port on a computer or laptop - USB hubs are not recommended as per Microchip's datasheet for the curiosity kit; failing to do so could cause headaches with trying to find the correct devices in the device manager during driver install. In Device manager look for other devices and set up as per the quick start guide.

X THIS IS NOT NEEDED AS THE TOOLCHAIN IS AUTOMATICALLY DOWNLOADED WHEN INITIALISING THE SDK. Start with toolchain: needs to be configured to the architecture and licensing etc. Architecture is RISCV. There is a version
- xpack-riscv-none-elf-gcc-13.2.0.2 - given by Hart software services, though Mastering linux for embedded design book doesn't recommend using toolchains
built by SoC providers as they are usually poorly configured. Recommend using it for learning.
The problem then moves to open source toolchains which in industry aren't really relied upon due to licensing and performance limitations.

X. NOT NEED AS PER ABOVE to install the prebuilt toolchain on linux host machine by extracting and decompressing it with (note that thes toolchains are for the beagle play arm soc and will have to be updated to suit pic64gx riscv toolchain):
bzip2 - d aarch64--glibc--stable-2024.02-1.tar.bz2
tar -xvf aarch64--glibc--stable-2024.02-1.tar

NOTE: pic64gx-hart-software-services has some baremetal examples in the core folders of the application folder

2. The PIC64GX SDK uses a specific version of python (between 3.8 - 3.12) upon initialisation to create the necessary files and folders for the workspace. Version 3.13 and above cannot (currently as at 10/09/26) be used due to a known dependency conflict. Rather than uninstalling any higher versions, which may cause dependency problems for other applications, a virtual environment (venv) can be setup so that the project only uses version 3.12, whilst other projects can still use version 3.13 and above where necessary.

2a. If not already done so, install python version 3.12 to c: drive from https://www.python.org/downloads/

2b. Create a python version 3.12 virtual environment, by opening CMD, and cd to beside th
e sdk folder (currently C:\\Users\\fende\\Desktop\\My_Projects\\PIC64GX_Projects). run
these commands in powershell:

    py -3.12 -m venv {venv_name}
2c. then:
    {path to sdk}\\{venv_name}\\Scripts\\activate.bat

2d. Point the project path to use the python 3.12 venv in environment variables. Then ope
n vs code, press ctrl+shift+p and "python: select interpreter: then choose the one at the
 venv location you just create; this will tell vscode you want your project to use the ve
rsion 3.12 virtual environment as the default python application for open .py files, circ
umnavigating any higher versions that might be install to c: drive.

2e. Ensure pip (use powershell for these):

    C:\\Users\\fende\\Desktop\\My_Projects\\PIC64GX_Projects\\venv_3.12\\Scripts\\pyth
on.exe -m ensurepip --upgrade

2f. check pip version:

    C:\\pic64gx-sdk\\venv312\\Scripts\\python.exe -m pip --version

2g. install pip:

    C:\\pic64gx-sdk\\venv312\\Scripts\\python.exe -m pip install --upgrade pip

2h. check against pic64gx requirements (IGNORE THIS UNLESSSDK ALREADY INSTALLED<, THOUGH
I WOULD SETUP THE VENV FIRST):

    C:\\pic64gx-sdk\\venv312\\Scripts\\python.exe -m pip install -r C:\\Users\\fende\\Des
ktop\\My_Projects   \\PIC64GX_Projects\\sdk\\scripts\\requirements\\requirements.txt

2j. then initialise (ignore these; it is just the cmd way to initiliase the sdk. easier t
o use the method below):

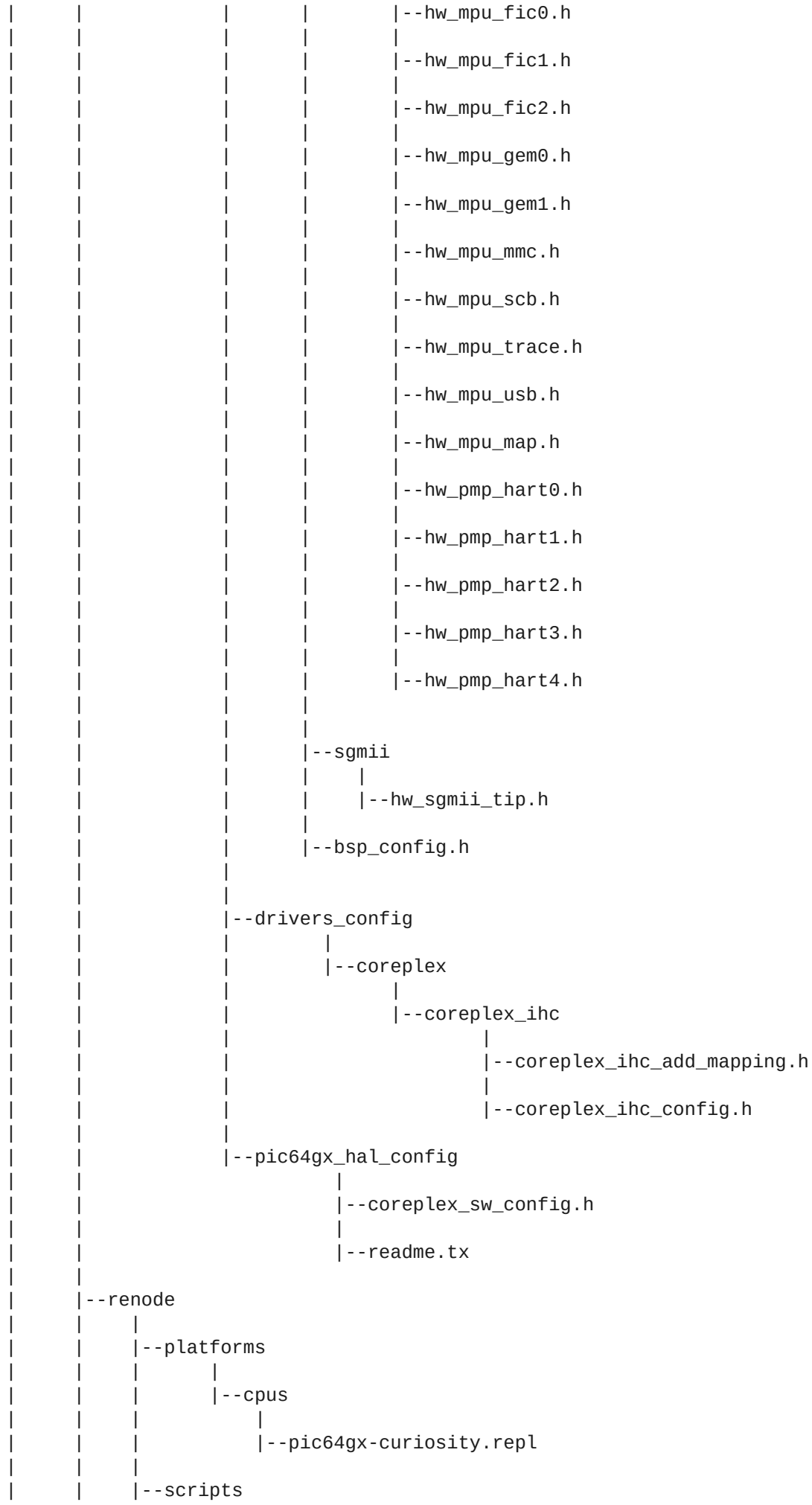
    cd C:\\Users\\fende\\Desktop\\My_Projects\\PIC64GX_Projects\\sdk

    C:\\pic64gx-sdk\\venv312\\Scripts\\python.exe .\\scripts\\init.py

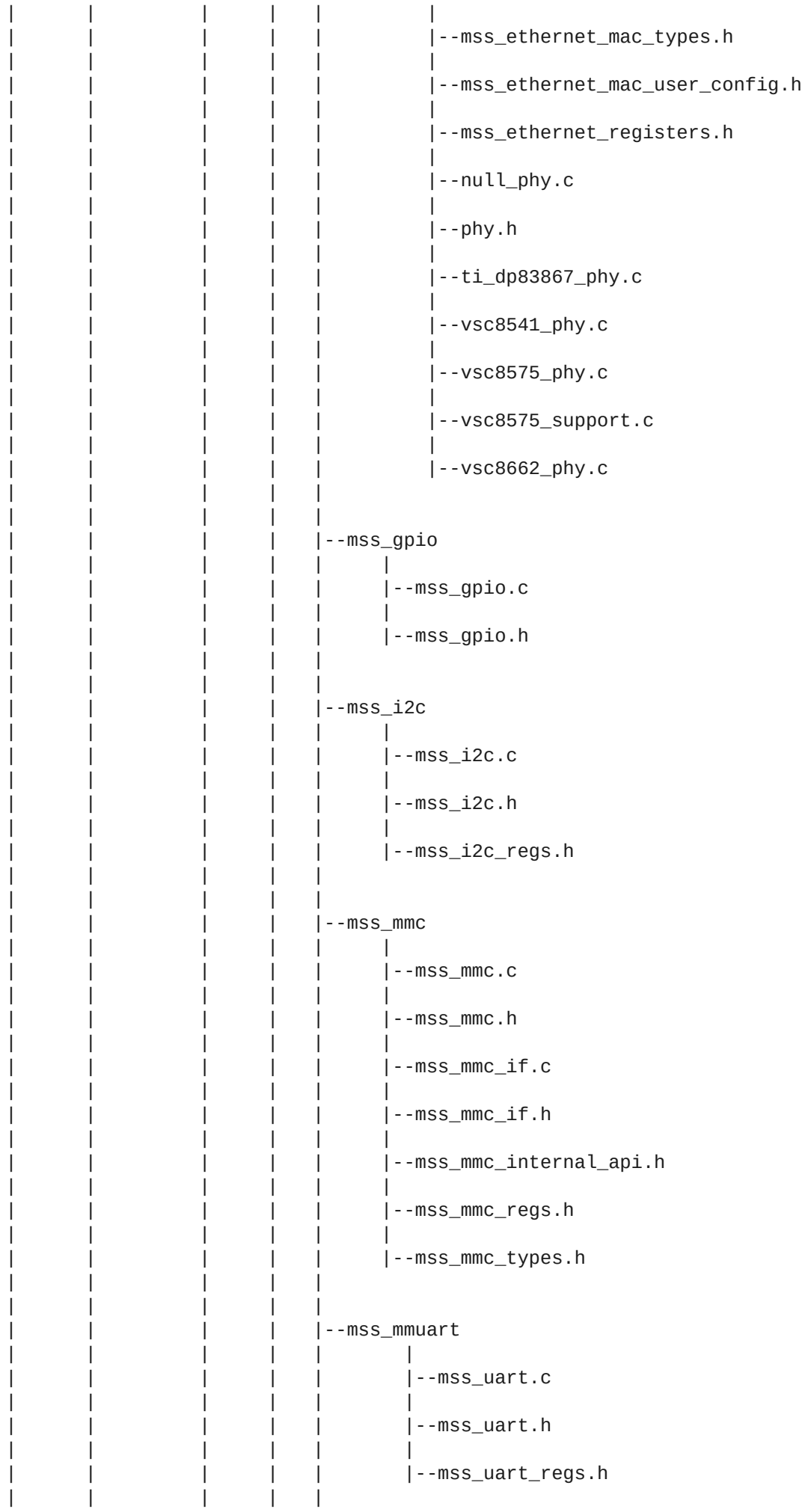
3. To set up the SDK properly, one should first locate the readme.md file in the pic64gx
sdk vscode extension folder (located at C:\Users\fende\.vscode\extensions\microchip.pic64
gx-sdk-extension-0.1.6). This readme has valuable information and instructional gifs on h
ow to initialise the sdk. NOTE that the sdk seems like it can be initialised in any folde
r, in which it will add the following folder structure to that folder (this may be incomp
lete due to writing it when I didn't realise I was having initialisation problems):

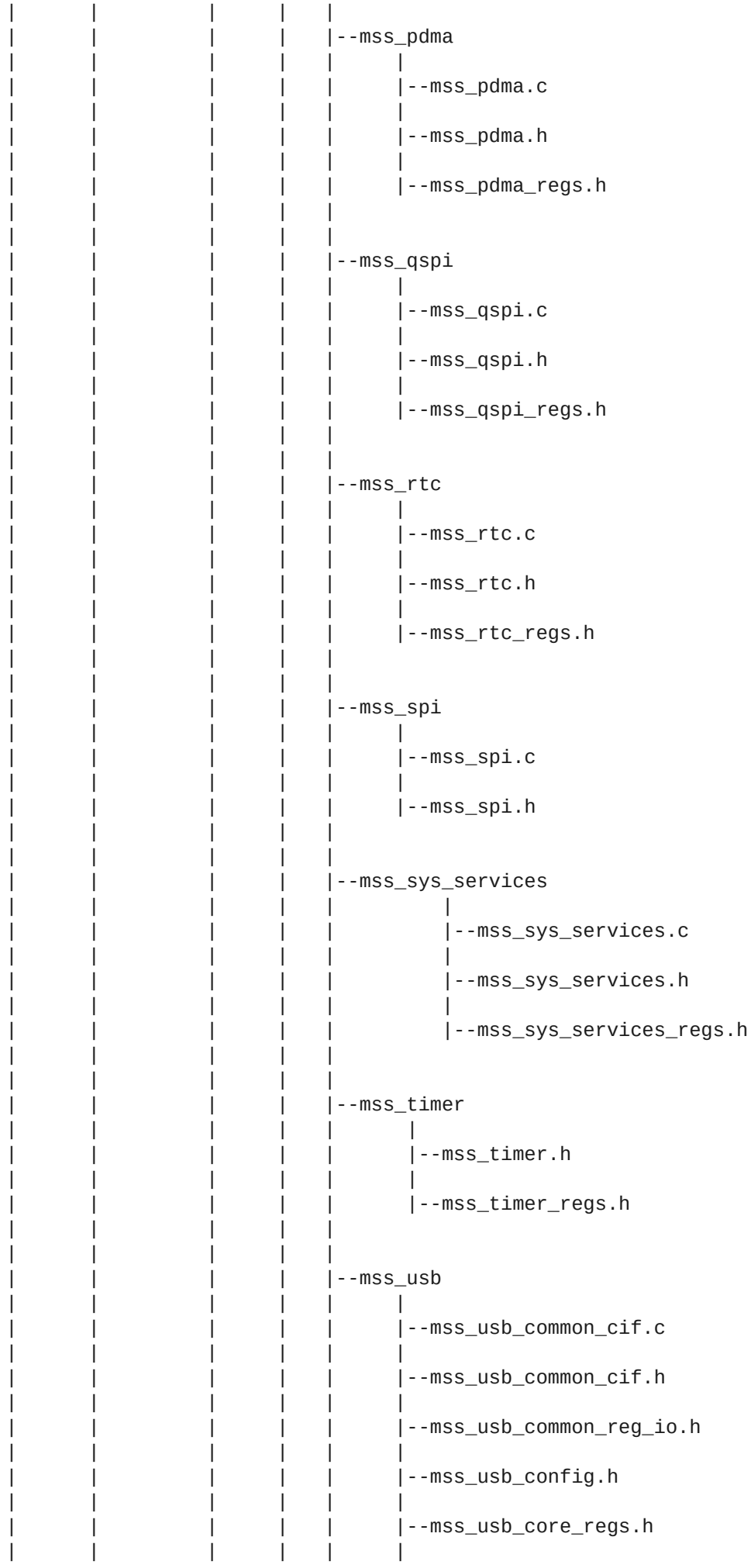
```
[Selected Folder]
    |
    |--MANIFEST            <<<<<< IMPORTANT: when opening a project, you have to select a
 folder containing this file
        |
    |--sdk
            |
            |--.vscode
            |     |
            |     |--c_cpp_properties.json
            |     |
            |     |--launch.json
            |     |
            |     |--settings.json
        |
```

```
    |
|--boards
    |     |
    |     |--pic64gx-curiosity-kit
    |     |             |
    |     |             |--bsp_config
    |     |             |       |
    |     |             |       |--clocks
    |     |             |       |    |
    |     |             |       |    |--hw_clk_ddr_pll.h
    |     |             |       |    |
    |     |             |       |    |--hw_clk_msscfm.h
    |     |             |       |    |
    |     |             |       |    |--hw_clk_mss_pll.h
    |     |             |       |    |
    |     |             |       |    |--hw_clk_sgmii_cfm.h
    |     |             |       |    |
    |     |             |       |    |--hw_clk_sgmii_pll.h
    |     |             |       |    |
    |     |             |       |    |--hw_clk_sysreg.h
    |     |             |       |    |
    |     |             |       |    |--hw_mss_clks.h
    |     |             |       |
    |     |             |       |
    |     |             |       |--ddr
    |     |             |       |   |
    |     |             |       |   |--hw_ddr_io_bank.h
    |     |             |       |   |
    |     |             |       |   |--hw_ddr_mode.h
    |     |             |       |   |
    |     |             |       |   |--hw_ddr_off_mode.h
    |     |             |       |   |
    |     |             |       |   |--hw_ddr_options.h
    |     |             |       |   |
    |     |             |       |   |--hw_ddr_segs.h
    |     |             |       |   |
    |     |             |       |   |--hw_ddrc.h
    |     |             |       |
    |     |             |       |
    |     |             |       |--general
    |     |             |       |    |
    |     |             |       |    |--hw_gen_peripherals.h
    |     |             |       |
    |     |             |       |
    |     |             |       |--io
    |     |             |       |   |
    |     |             |       |   |--hw_hsio_mux.h
    |     |             |       |   |
    |     |             |       |   |--hw_mssio_mux.h
    |     |             |       |   |
    |     |             |       |   |--hw_mssio_mux_alternate.h
    |     |             |       |
    |     |             |       |
    |     |             |       |--memory_map
    |     |             |       |       |
    |     |             |       |       |--hw_apb_split.h
    |     |             |       |       |
    |     |             |       |       |--hw_cache.h
    |     |             |       |       |
    |     |             |       |       |--hw_memory.h
    |     |             |       |       |
    |     |             |       |       |--hw_mpu_crypto.h
    |     |             |       |       |
```


|--hw_mpu_fic0.h
|--hw_mpu_fic1.h
|--hw_mpu_fic2.h
|--hw_mpu_gem0.h
|--hw_mpu_gem1.h
|--hw_mpu_mmc.h
|--hw_mpu_scb.h
|--hw_mpu_trace.h
|--hw_mpu_usb.h
|--hw_mpu_map.h
|--hw_pmp_hart0.h
|--hw_pmp_hart1.h
|--hw_pmp_hart2.h
|--hw_pmp_hart3.h
|--hw_pmp_hart4.h
|--sgmii
|--hw_sgmii_tip.h
|--bsp_config.h
|--drivers_config
|--coreplex
|--coreplex_ihc
|--coreplex_ihc_add_mapping.h
|--coreplex_ihc_config.h
|--pic64gx_hal_config
|--coreplex_sw_config.h
|--readme.tx
|--renode
|--platforms
|--cpus
|--pic64gx-curiosity.repl
|--scripts

```
|     |          |
|     |          |--macros.resc
|     |          |
|     |          |--pic64gx-curiosity.resc
|     |          |
|     |          |--pic64gx-curiosity-platform.resc
|     |
|     |--Kconfig
|
|--cmake
|    |
|    |--common.cmake
|    |
|    |--cross-compiler-gcc-tc.cmake
|    |
|    |--defaultPreset.json
|    |
|    |--preload.cmake
|    |
|    |--README.md
|
|--images
|    |
|    |--add-breakpoint.gif
|    |
|    |--breakpoint-reached.png
|    |
|    |--build-project.gif
|    |
|    |--create-project.gif
|    |
|    |--init-finished.png
|    |
|    |--init-new.gif
|    |
|    |--start-debugging.gif
|
|--platforms
|      |
|      |--mss
|      |   |
|      |   |--platform
|      |          |
|      |          |--drivers
|      |          |     |
|      |          |     |--mss
|      |          |     |   |
|      |          |     |   |--mss_can
|      |          |     |   |     |
|      |          |     |   |     |--mss_can.c
|      |          |     |   |     |
|      |          |     |   |     |--mss_can.h
|      |          |     |   |
|      |          |     |   |
|      |          |     |   |--mss_ethernet_mac
|      |          |     |   |         |
|      |          |     |   |         |--mss_ethernet_mac.c
|      |          |     |   |         |
|      |          |     |   |         |--mss_ethernet_mac.h
|      |          |     |   |         |
|      |          |     |   |         |--mss_ethernet_mac_regs.h
|      |          |     |   |         |
|      |          |     |   |         |--mss_ethernet_mac_sw_cfg.h
```


|--mss_ethernet_mac_types.h
|--mss_ethernet_mac_user_config.h
|--mss_ethernet_registers.h
|--null_phy.c
|--phy.h
|--ti_dp83867_phy.c
|--vsc8541_phy.c
|--vsc8575_phy.c
|--vsc8575_support.c
|--vsc8662_phy.c
|--mss_gpio
|--mss_gpio.c
|--mss_gpio.h
|--mss_i2c
|--mss_i2c.c
|--mss_i2c.h
|--mss_i2c_regs.h
|--mss_mmc
|--mss_mmc.c
|--mss_mmc.h
|--mss_mmc_if.c
|--mss_mmc_if.h
|--mss_mmc_internal_api.h
|--mss_mmc_regs.h
|--mss_mmc_types.h
|--mss_mmuart
|--mss_uart.c
|--mss_uart.h
|--mss_uart_regs.h


|--mss_pdma
|--mss_pdma.c
|--mss_pdma.h
|--mss_pdma_regs.h
|--mss_qspi
|--mss_qspi.c
|--mss_qspi.h
|--mss_qspi_regs.h
|--mss_rtc
|--mss_rtc.c
|--mss_rtc.h
|--mss_rtc_regs.h
|--mss_spi
|--mss_spi.c
|--mss_spi.h
|--mss_sys_services
|--mss_sys_services.c
|--mss_sys_services.h
|--mss_sys_services_regs.h
|--mss_timer
|--mss_timer.h
|--mss_timer_regs.h
|--mss_usb
|--mss_usb_common_cif.c
|--mss_usb_common_cif.h
|--mss_usb_common_reg_io.h
|--mss_usb_config.h
|--mss_usb_core_regs.h

```
|      |         |     |   |     |--mss_usb_device.c
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_device.h
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_device_cif.c
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_device_cif.h
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_device_hid.c
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_device_hid.h
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_device_msd.c
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_device_msd.h
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_device_reg_io.h
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_device_vc.c
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_device_vc.h
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_host.c
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_host.h
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_host_cif.c
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_host_cif.h
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_host_hid.c
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_host_hid.h
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_host_msc.c
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_host_msc.h
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_host_reg_io.h
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_regs.h
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_std_def.h
|      |         |     |   |
|      |         |     |   |
|      |         |     |   |--mss_watchdog
|      |         |     |   |        |
|      |         |     |   |        |--mss_watchdog.c
|      |         |     |   |        |
|      |         |     |   |        |--mss_watchdog.h
|      |         |     |   |
|      |         |     |   |
|      |         |     |   |--pf_pcie
|      |         |     |         |
|      |         |     |         |--pf_pcie.c
|      |         |     |         |
|      |         |     |         |--pf_pcie.h
|      |         |     |         |
|      |         |     |         |--pf_pcie_regs.h
|      |         |     |         |
|      |         |     |         |--pf_pcie_types.h
|      |         |     |
|      |         |     |--pf_pcie
```

```
|      |         |
|      |         |--hal
|      |         |
|      |         |--pic64gx_hal
|      |         |
|      |         |--CMakeLists.txt
|      |         |
|      |         |--Kconfig
|      |         |
|      |         |--LICENSE.md
|      |
|      |--CMakePresets.json
|
|--scripts
|
|--tools
|
|--gitignore
|
|--CMakeLists.txt
|
|--CMakePresets.json
|
|--Kconfig
|
|--README.md
```

4. To initialise the sdk, make sure vscode is opened at the folder you want the workspace (currently: C:\\Users\\{user_name}\\Desktop\\My_Projects\\PIC64GX_Projects) - this wil l be where the workspace is created!!! IMPORTANT: The venv 3.12 and its pip install must be setup and activated in this same folder PRIOR to initialising the SDK, as well as that folder being selected as the interpreter pathway in vscode after pressing Ctrl+shift+P. Go into environment variables and check that these paths are correct. Also close any othe r instances of the path location that might be open in file explorer to avoid access conf licts during initialisation.

Select "initialise sdk" in the bottom left blue/purple tool bar (will vary depending on m achine/ vs code setup) of vscode, and select the project folder you just created (\[selec ted folder] in the tree above), or press ctrl+shift+P and search for "initialise sdk". Th is will clone necessary files from GitHub into that folder, including the MANIFEST file. There must be no errors on the vscode output window. if there is an error regarding certi ficates whilst downloading renode, open cmd and type the following:

Install/Update certifi:
    C:\Users\{user_name}\Desktop\My_Projects\PIC64GX_Projects\venv_3.12\Scripts\python.ex e -m pip install --upgrade certifi

Run install certificates script:
    C:\Users\{user_name}\Desktop\My_Projects\PIC64GX_Projects\venv_3.12\Scripts\python.ex e -m pip install pip-system-certs

re-run initialisation (or do this automatically in vscode by clicking on "initialise sdk")
    C:\Users\{user_name}\Desktop\My_Projects\PIC64GX_Projects\venv_3.12\Scripts\python.ex e sdk/scripts/init.py

Create a new project by clicking on the PIC64GX SDK icon in the toolbar on the left side of the vscode screen, and then "Create SDK Project" in the panel that pops up (NOT the "c reate new project" button in "Open Configurator", as this is specifically for creating pr ojects using AMP and will be configured belowafter a global project has been created. Ent er a project name and a project directory int he window that pops up. NOTE: I would not s uggest manually tryign to find a project folder; rather, TYPE in the same name as the pro ject name or a name that is relevant to the project. In the bottom "Configure Project" wi

ndow, "Boards Software Configuration", "Linker Script", "MSS Drivers", and "Platform" sho
uld automatically appear. Scroll down and click the "Create Project" button; a new folder
 will be created in the workspace folder alongside the sdk with the following structure:

```
    workspace folder (PIC64GX_Projects)
            |
            |--.vscode
            |
            |--boards
            |
            |--{new project folder}
            |        |
            |        |--temp (used for setup and auto deleted when not needed)
            |        |
            |        |--src
            |        |
            |        |--CMakeLists
            |        |
            |        |--proj.conf
            |
            |--modules
            |
            |--platforms
            |
            |--sdk
            |
           etc
```

Once the SDK has been initialised, selecting AMP and DDR from the explorer menu of vscode
 under the configurator dropdown should now open a window with selectable options rather
than not doing anything. NOTE that Linux and Zephyr are the operating systems that can be
 chosen for each context with Zephyr being specifically an RTOS (Real Time Operating Syst
em) - more information on this RTOS is available from Microchip; the important thing to n
ote is that one context can run Linux whilst the other runs Zephyr. AMP - Asynchronous Mu
lti Processing - is the means by which they communicate with each other, as opposed to si
ngle context operation which is SMP (fact check this).
With AMP not selected, context A is automatically selected as processor, with context B b
eing locked out of operation. Selecting AMP unlocks context B, but before a project can b
e created a few settings rules need to be implemented otherwise vscode will throw some er
rors:
    First, atleast 1 hart core has to be dedicated to context B.
    Secondly, context B also needs a main form of memory set up for it under the Memory M
ap/ DDR options of the side bar panel.

It is therefore recommend to use the following settings for AMP operation:
    AMP selected
    Context A = Linux
    Context B = Zephyr
    U54_1,2,3 = Context A
    U54_4 = Context B
    SPI1 = Context A
    Inter-Hart Comms = Context A and B
    Video pipeline = context A

    All other options left as default

IMPORTANT: In order to use click board plugged into the mikrobus socket, you need to ensu
re its SPI1 port has a context otherwise it will be disable during build time. After the
files have been generated in the project directory, check that its status is "okay" - not
 "disabled" - in the context.dtso file, mentioned below.

Once these settings have been chosen, pressing create project in the configurator window will generate a MANIFEST file in the selected project location. I would suggest selecting the project folder created before when creating a new project. Once this manifest file has been created, a message "Default DTS File:pic64gx-curiosity-kit-default.dts" and "Included DTSI File:pic64gx_default.dtsi" will appear above the open project/ create new project buttons of the configurator window. Pressing the "Generate" button will now generate the dtsi and dtb device tree files for AMP use. NOTE that the output will automatically choose the path to the installed HSS directory. I would recommend choosing a specific project folder that was created after sdk initialisation. If the HSS folder automatically appears in the top of the vscode window and doesn't seem to allow for a folder change, you will need to click the toolbar in the same area with the + symbol; after this, file explorer should automatically open when the generate button is clicked. Also NOTE that the configurator tab needs to be selected in the vscode window (treat it as if it is python file you wish to view) in order for the AMP/DDR settings to show.

If vscode is closed, or if the folder root selection is changed, you will need to REOPEN THE FOLDER WHERE THE MANIFEST FILE WAS CREATED - BEFORE YOU CAN ACCESS AMP AND DDR SETTINGS. PIC64GX Configurator - VSCode Extension has more information on this.

The following files will be generated, assuming amp is selected (if it isn't selected only the dtbo, dtso and overlay files for context A will be generated.

```
{selected output folder}
      |
          |--context_a.dtbo
          |
          |--context_a.dtso
          |
          |--context_a.overlay
          |
          |--context_b.dtbo
          |
          |--context_b.dtso
          |
          |--context_b.overlay
          |
          |--global.yaml
```

6\. The above files are used to configure the device for boot (refer to Mastering Embedded Linux Development book. page 54 onwards for instruction on how to use). Opening the global.yaml file will give the OS contexts as well as configured peripherals.

NOTE: the MANIFEST and global.yaml files are used to restore project settings whenever a project is opened. check PIC64GX Configurator - VSCode Extension file for more info.

Locate the context_a.dtso file (or whatever context spi1 was given) and replace the spi1status line with the following:

```
&spi1 {
 status = "okay";
   spidev@0 {
      #address-cells = <1>;
      #size-cells = <1>;
      compatible = "microchip,pic64gx-spidev";
      spi-max-frequency = <20000000>;
      reg = <0>;
   };
};
```

DEVICE TREE BASICS:

"A device tree is a flexible way of defining the hardware components of a computer system . It is just static data, not executable code. Usually the device tree is loaded by the b ootloader and passed to the kernel, although it is possible to bundle the device tree wit h the kernel image itself to cater to bootloaders that are not capable of loading them se parately.

The format is derived from Sun Microsystems bootloader known as OpenBoot., which was form alised as the Open Firmware Specifiaction (IEEE Standard IEEE1275-1994). https://www.dev icetree.org has more information.

Device tree represents a computer system as a collection of components joined together in a hierarchy. Every device tree begins with a root node represented by a forward slas,/, which contains subsequent child nodes describing the hardware of the system. Each node of has a name and contains several properties in the form name = "value". Example:

```
/dts-v1;
/{
    model = "TI AM335x BeagleBone";
    compatible= "ti, am33xx";
    #address-cells = <1>;
           #size-cells = <1>;

           cpus {
             #address-cells = <1>;
      #size-cells = <0>;
             cpu@0 {
                compatible = "arm, cortex-a-8";
                device\_type = "cpu";
                reg = <0>;
             }
           }
           memory@80000000 {
            device\_type = "memory";
            reg = <0x80000000 0x20000000>; /\* 512 MB \*/
        };
```

Here we have a root node that contains a cpus node and a memory node. the cpus node conta ins a single cpu node named cpu@0. The names of these often include an @ followed by an a ddress that distinguishes the node from other nodes of the same type. @ is required if th e node has a reg property.
Both the root and CPU nodes have a compatible property. The Linux kernel uses this proper ty to find a matching device driver by comparing it with the strings that are exported by each device driver in an of_device_id structure (check chapter 11 for more)

IMPORTANT: it is a convention that the value of the compatible property is composed of a manufacturer name and a component name to reduce confusion between similar devices made b y different manufacturers, hence ti, am33xx and arm, cortex-a8. It is also quite common t o have more than one value for the compatible property when there is more than one driver thatcan handle this device. They are listed with the most suitable mentioned first."

BARE METAL APPLICATIONS
Each project should have the following structure, which is used to run c source and heade r files on each hart:

[Selected Folder] (should be C:\Users\fende\Desktop\My_Projects\PIC64GX_Projects\{projec t_name} if steps above were followed)

```
      demo
    |
    |--CMakeLists.txt
    |
    |--proj.config
```

```
        |
    |--src
            |
            |--application
               |
               |--hart0
           |    |
           |    |--e51.c
           |
               |--hart1
           |    |
           |    |--u54_1.c    <<<<<< demo will edit this file
               |
               |--hart2
           |    |
           |    |--u54_2.c
           |
               |--hart3
           |    |
           |    |--u54_3.c
           |
               |--hart4
           |    |
           |    |--u54_4.c
           |
               |--inc
               |
               |--common.h
```

Navigate to the hart0_e51.c file and ensure it has a basic hello world application. if no ne exists, use the following.

```
#include "pic64gx_hal/mss_hal.h"
#include "mss_mmuart/mss_uart.h"


const uint8_t g_message[] = "\r\n Hello, World!\r\n";

void e51(void)
{
    (void) mss_config_clk_rst(MSS_PERIPH_MMUART0, (uint8_t) 1, PERIPHERAL_ON);

    MSS_UART_init(&g_mss_uart0_lo,
    MSS_UART_115200_BAUD,
    MSS_UART_DATA_8_BITS | MSS_UART_NO_PARITY | MSS_UART_ONE_STOP_BIT);

    /* Message on uart0 */
    MSS_UART_polled_tx(&g_mss_uart0_lo, g_message, sizeof(g_message));

    while (1U)
    {

    }
}
```

Navigate to hart1/ u54_1.c file, select all and replace the txt with the following applic ation that will flash an led on and off on the PIC64GX curiosity kit (original file can b e found at https://developerhelp.microchip.com/xwiki/bin/view/products/mcu-mpu/64bit-mpu/ intro-to-pic64gx-development/):

```
    #include <stdint.h>
    #include "pic64gx_hal/mss_hal.h"
```

```
#include "mss_gpio/mss_gpio.h"

void delay(uint8_t seconds){
       for (uint64_t i = 0; i < (seconds * 11000000U); ++i) {
         __asm__("sll x0, x0, x0"); // Dummy instruction for delay
       }
}

void u54_1(void)
{
        // Enable clock for GPIO1 peripheral
        (void)mss_config_clk_rst(MSS_PERIPH_GPIO1, (uint8_t) 1, PERIPHERAL_ON);

        // Initialize GPIO1 lower half
        MSS_GPIO_init(GPIO1_LO);

        // Configure pin 2 as output
        MSS_GPIO_config(GPIO1_LO, MSS_GPIO_2, MSS_GPIO_OUTPUT_MODE);

        // Blink loop
        while (1u)
        {
           delay(1);
           MSS_GPIO_set_output(GPIO1_LO, MSS_GPIO_2, 1u); // LED ON
           delay(1);
           MSS_GPIO_set_output(GPIO1_LO, MSS_GPIO_2, 0u); // LED OFF
        }
     }
```

NOTE: if we look at CMakeLists.txt and proj.conf contents, we get insight into how CMake created this file structure as well as the following configurations:

In CMakeLists.txt:

```
include(${WORKSPACE}/sdk/cmake/common.cmake)       =      pulls in Microchip's SDK build infrastructure

sdk_project(PIC64GX_Projects)         =     creates the SDK project

target_sources(${PROJECT_NAME} PUBLIC
    .../hart0/e51.c
    .../hart1/u54_1.c
    .../hart2/u54_2.c                 =    explicitly adds one source file for each hart
    .../hart3/u54_3.c
    .../hart4/u54_4.c
)
```

In proj.config:

```
CONFIG_PIC64GX_HAL_LAST_HART=4             =     corresponds to 5 hart setup

CONFIG_SGMII_SUPPORT=y
CONFIG_DDR_SUPPORT=y
CONFIG_MSSIO_SUPPORT=y                =     generated project configured for the MSS platform and MMUART amongst other things
CONFIG_PLATFORM_MSS=y
CONFIG_PLATFORM=y
CONFIG_MSS_MMUART=y
```

(these lines may not be there due to discrepancy with set up. left in for the sake of lea
rning how to implement memory management)
CONFIG_MEMORY="lim"
CONFIG_APPLICATION_STACK_SIZE=8192
CONFIG_APPLICATION_HEAP_SIZE=8192 = application has be
en configured with specific memory resources rather than us having to invent them case
CONFIG_APPLICATION_RAM_SIZE=32768

We need to append CONFIG_MSS_GPIO=y to the bottom line of the proj.conf file, and save it
:

...
CONFIG_PLATFORM=y
CONFIG_MSS_MMUART=y
CONFIG_MSS_GPIO=y

Now in vscode, make sure serial monitor extension has been downloaded and installed. Clic
k on serial monitor tab in the output window at the bottom of the screen, and click the b
lue "+Open Additional Monitor" button 2 times until 3 serial monitors have been opened (s
ometimes one of them will become hidden and will need to be found by clicking either the
serial monitor(#) text or expanding the window if it was minimised). Select the com ports
associated with the MCHP-Debug-Ports in Device Manager>>>>Ports (COM and LPT) (Currently
, COM9, COM10 and COM11). Ensure Baud rate and other settings are correct by clicking on
the cog icon for each monitor if they aren't displayed (the com port settings can be foun
d by right clicking on com port one in Device Manger, Properties and the Port Settings ta
b). Click "Start Monitoring" for each of the three monitors.

BUILDING

6.The next step is to run build in the pic64gx plugin in the vscode window. For a build,
the correct board needs to be selected from the SDK - currently: C:\Users\fende\Desktop\
My_Projects\PIC64GX_Projects\boards\pic64gx-curiosity-kit. NOTE that using the one in Ha
rt software Services will result in a failed build. A project folder also needs to be sel
ected; choose one that was created with the "Create SDK project" button after the SDK was
initialised.

A build folder will be created in the workspace with the following structure:

```
workspace folder (PIC64GX_Projects)
        |
        |--.vscode
        |
        |--boards
        |
        |--build
        |    |
        |    |--board
        |    |
        |    |--CMakeFiles
        |    |
        |    |--.config
        |    |
        |    |--.ninja_deps
        |    |
        |    |--.ninja_log
        |    |
        |    |--autoconf.h
        |    |
```

```
|    |--build.ninja
|    |
|    |--cmake_install.cmake
|    |
|    |--CMakeCache.txt
|    |
|    |--compile_commands.json
|    |
|    |--{project_name}.bin
|    |
|    |--{project_name}.elf
|    |
|    |--{project_name}.lst
|    |
|    |--{project_name}-mine.map
|    |
|    |--generated-linker.ld
|
|--{new project folder}
|        |
|        |--temp (used for setup and auto deleted when not needed)
|        |
|        |--src
|        |
|        |--CMakeLists
|        |
|        |--proj.conf
|
|--modules
|
|--platforms
|
|--sdk
|
etc
```

RUNNING AND DEBUGGING

Once we have our {project_name}.elf file, we can run the application on the PIC64GX and debug it by clicking on the "Run and Debug" icon in the left side panel toolbar of vscode. This will open a new panel with a green play button at the top. Upon clicking this button a message saying "path to the elf file relative to the workspace etc build/*.elf (Press 'Enter' to confirm or 'Escape' to cancel)" and you will be prompted to enter the path into a text box. Type \build\{project_name}.elf and press enter.
When prompted for a debug probe, select PIC64GX. Debug Console should output a bunch of yellow text, and a toolbar with stop, and step over etc button should pop up at the top of vscode.

Switch the output window back to Serial Monitor and make sure that all three monitors are running. One of the com ports should out put Hello World, as the application source in the hart0/e51.c file dictates. An orange LED should start flashing on the PIC64GX curiosity board roughly every second, as dictated by the source in the hart1/u54_1.c file.